Kael Hufford – CIT360 – Assignment 0 – Sorting Algorithm Performance Analysis

1. Method

a. To do this, integer arrays were generated at sized ranging from 0 to 200,000 elements in increments of 100. Each sorting algorithm received a copt of the same original array. Execution time was measured using System.nanoTime() and converted to milliseconds. Five sorting tasks ran at the same time using a fixed thread pool. Each array size was measured once, then they were saved before exporting to a CSV.

2. Sorts Being Used

a. **Bubble Sort**
   i. *Description* – Bubble Sort finds adjacent values and swaps them whenever they are out of order. It continuously does this and cycles through until the given list is completely sorted.
   ii. *Expected growth* – $O(n^2)$
   iii. *Resulting Growth* – $O(n^2)$

b. **Selection Sort**
   i. *Description* – Selection Sort finds the smallest remaining value and swaps it with the value in the next unsorted position.
   ii. *Expected growth* – $O(n^2)$
   iii. *Resulting Growth* – $O(n^2)$

c. **Insertion Sort**
   i. *Description* – Insertion sort takes each value and inserts it into the correct position in the already sorted portion.
   ii. *Expected growth* – $O(n^2)$
   iii. *Resulting Growth* – $O(n^2)$

d. **Quick Sort**
   i. *Description* – Quick Sort selects a pivot and partitions the array so values smaller than the pivot appear before it and larger values appear after it. It then recursively sorts the two partitions.
   ii. *Expected growth* - $O(n * log(n))$
   iii. *Resulting Growth* - $O(n * \log(n))$

e. **Merge Sort**
   i. *Description* – Merge sort continues to split the array into halves until it can't. Then it sorts the smallest parts merging them together until the array is completely put together, and hence, sorted.
   ii. *Expected growth* - $O(n * \log(n))$
   iii. *Resulting Growth* - $O(n * \log(n))$

3. Results

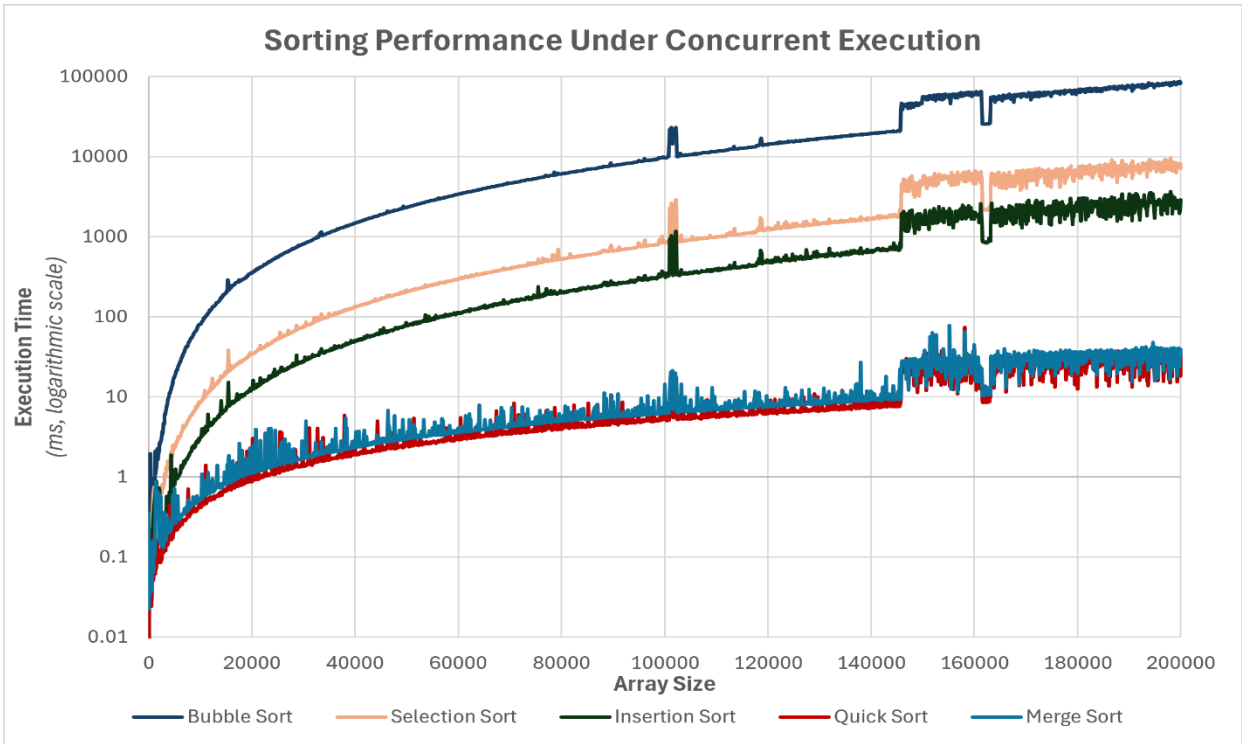


Figure 1. Execution time of 5 sorting algorithms from 0 to 200,000 elements. The Y-Axis used a logarithmic scale.

- Findings of the sorting:
    - Bubble Sort has the greatest runtime.
    - Selection and Insertion sort showed quadratic growth.
    - Quick Sort and Merge Sort remained considerably faster.
    - The general results agree and support the expected complexities.
    - The logarithmic scale was used because the runtimes differed by several orders of magnitude.

4. Limitations

- The sorts ran concurrently and used the same available CPU resources.
- Background applications and processor temperature may have affected results.
- Measurements may contain noise because each size was measured once.
- All timing measurements were collected while the sorting algorithms ran concurrently.

5. Additional Sources Used

- Basic threading – W3Schools: Java Threads
  - https://www.w3schools.com/java/java_threads.asp
- Lambda in Java – W3Schools: Java Lambda Expressions
  - https://www.w3schools.com/java/java_lambda.asp
- File I/O in Java – W3Schools: Java Write to Files & Java File Handling
  - https://www.w3schools.com/java/java_files_write.asp
  - https://www.w3schools.com/java/java_files.asp
- Multithreading: Oracle: Executor, ExecutorService, Future
  - https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Executor.html
  - https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ExecutorService.html
  - https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Future.html
  - https://docs.oracle.com/javase/tutorial/essential/concurrency/